Enterprise AI Integration Engineer

A 14-week self-study roadmap — from LLM fundamentals to production-grade enterprise AI integration

Who this is for: an experienced backend / distributed-systems engineer (Java, Kafka, event-driven architecture) who wants to integrate LLMs into large enterprise systems — with strong fundamentals on how the models actually work, building on existing JVM/enterprise strengths.
Time: ~10–15 hrs/week · **Duration:** ~14 weeks · **Outcome:** a portfolio that shows *production-grade* enterprise AI integration.

Your edge — read this first

The scarce skill isn't "can call an LLM API." It's: *drop AI into a large company's existing estate — legacy services, security reviews, data-residency rules, SLAs, cost ceilings, thousands of users — without it falling over.* That's a distributed-systems problem with an LLM in the loop, and a strong backend engineer already owns the hard half. This plan makes you dangerous at the LLM half and shows you how the two connect.

Deliberate choices:

- **Polyglot, JVM as home base — not Spring-only.** JVM strengths (Spring AI, LangChain4j) are an advantage, so use them where they fit — but don't be limited to them. Python is the AI ecosystem's lingua franca (most new tools, SDKs, and papers land there first), so get fluent enough to use it when it's the better tool (embeddings, evals, quick prototypes, MCP servers). Pick the stack per project: JVM when integrating into JVM systems, Python when the ecosystem gives you leverage. Comfort in both is exactly what makes an enterprise AI integrator valuable.
- **Enterprise cloud, not just raw APIs.** Target **Azure OpenAI** and/or **AWS Bedrock** — how real companies consume models (private networking, data not used for training, compliance). Pick one as your primary (see last page). These are language-agnostic — Java, Python, or plain HTTP all work.
- **Every project is an *integration***, not a chatbot demo. Existing API, existing data, existing Kafka topic — AI added to it.

Phase 0 — How the model works, so you can debug it

Weeks 1–2

Strong fundamentals matter: when an LLM misbehaves in prod, the people who understand the mechanism are the ones who fix it. Conceptual, not math-heavy.

UNDERSTAND

- ☐ Next-token prediction; tokenization (run a tokenizer, watch cost = tokens); embeddings as vectors
- ☐ The transformer conceptually: attention = "which prior tokens matter." Be able to explain it to a coworker
- ☐ Training stages: pretraining → instruction tuning → RLHF — *why* chat models behave as they do

- ☐ Map each behavior to a cause: hallucination, context-window limits, sampling (temperature, top-k, top-p), latency, cost — all fall out of the mechanism
- ☐ Watch: Karpathy "Intro to LLMs"; 3Blue1Brown transformer series; Jay Alammar "Illustrated Transformer"
- ☐ Light ML context (intuition only): what training/loss/gradient descent mean; where LLMs sit vs classical ML

> **Build:** `00-token-cost-lab` — a small utility that tokenizes text, reports token counts, and computes per-request cost for your target model. Build it in whichever language you want to warm up in (Python's `tiktoken` makes it a 20-line script; doing it in Java teaches you more about what's happening). Trivial either way, but it makes tokens/cost real — and cost is an enterprise concern from day one.

Phase 1 — Talking to models from the JVM

Weeks 3–4

UNDERSTAND

- API primitives: messages, system prompt, temperature, streaming; system vs user turns
- Prompting patterns and *why* they work: in-context learning, chain-of-thought, structured output

> **Side quest (one evening):** run an open-source model locally with **Ollama** (Llama or Mistral) and hit it from code. Cheap way to build the closed-vs-open judgment enterprises ask about (cost, data residency, self-hosting trade-offs) — and it makes the cloud APIs less magic.

> **Build:** `01-spring-ai-service` — a **Spring Boot** service exposing a REST endpoint backed by an LLM (Azure OpenAI or Bedrock via Spring AI). Streaming responses, config-driven model/temperature, a health check. This is your "hello world" as a *service*, the way you'd actually ship it.

Phase 2 — Structured output & tool use = system integration

Weeks 5–6

This is where a backend background makes you fast. Tool calling is just "the model asks your service to do something" — an integration pattern you already know.

UNDERSTAND

- The tool-use loop: model requests, *you* execute, feed result back
- Structured output with schema validation + retry
- Failure modes (bad JSON, wrong tool, timeouts, rate limits) and enterprise-grade handling: retries, circuit breakers, fallbacks — your wheelhouse

> **Build:** `02-tool-service` — an LLM service (Spring AI / LangChain4j) that calls **2–3 real internal-style APIs** as tools (e.g. a lookup service, a database query, a calculation). Add resilience: timeout, retry, fallback model. Treat it like a service you'd put behind a real SLA.

Phase 3 — RAG over enterprise data

Weeks 7–9

The #1 enterprise AI use case: "answer questions over *our* documents/data." Where data-plumbing skills shine.

UNDERSTAND

- Embeddings & similarity search — and when they fail
- Chunking (matters more than people think); vector vs keyword (BM25) vs hybrid; reranking; metadata filtering
- Failure modes (retrieval misses, hallucinated citations) and how to detect them
- Enterprise angles: access control on retrieval (users must only see docs they're allowed to), PII, freshness

> **Build:** `03-enterprise-rag` — RAG Q&A over a realistic corpus using a JVM-friendly vector store (**pgvector** via Postgres, or Elasticsearch — infra you likely already run) and Spring AI / LangChain4j. **With citations and per-user access filtering.** Pick a corpus you know well, so you can tell when it's wrong.

Phase 4 — Evals, cost & observability

Weeks 10–11

The part that separates a demo from something you can put in front of a CISO and a CFO.

UNDERSTAND

- Eval sets; grading: exact match, rubric, LLM-as-judge + its biases
- Logging every call (input, output, tokens, latency, cost); tracing across services
- Guardrails: prompt-injection awareness, output filtering, PII redaction

> **Build:** an eval harness for project 03 (30–50 real Q&A, a grader, a metric you *improved*), plus **observability**: structured logs + metrics (Micrometer/OpenTelemetry → your existing stack) for token cost and latency per request. Now you can say "N% accuracy, $X per 1k requests, p95 latency Y" — the numbers enterprises decide on.

Phase 5 — Agents, orchestration & event-driven AI

Weeks 12–13

Where a Kafka/event-driven background becomes a superpower most AI people don't have.

UNDERSTAND

- Agent patterns: ReAct, planning, reflection, human-in-the-loop; when multi-agent helps vs hype
- **MCP (Model Context Protocol)** — the emerging standard for exposing tools/data to any model

> **Side quest (half a day):** a taste of **multimodal** — send an image (screenshot, scanned invoice, chart) to a vision-capable model and extract structured data from it. Document-understanding is a real enterprise use case; you don't need a full phase, just to know it exists and how it's called.

Build (pick one, both play to backend strengths):

- `04-event-driven-ai` — a **Kafka-based** pipeline where events trigger LLM processing (e.g. incoming support tickets → classify/route/draft reply → emit result event). Streaming + AI is a real differentiator.
- `04-agent` — a multi-step agent (research → cited report, or triage issues → draft fixes) built with LangChain4j / Spring AI, plus a small **MCP server** exposing one of your tools.

Phase 6 — Ship & position

Week 14

- ☐ Deploy your best project as a real service (container + your usual pipeline)
- ☐ READMEs with architecture diagrams, eval results, cost/latency numbers, security notes
- ☐ 1–2 blog posts — e.g. "Adding RAG to a Spring Boot service without leaking data across tenants." Enterprise-integration content stands out because almost no one writes it
- ☐ Reframe your title: **"Backend engineer specializing in enterprise AI integration."** Bring an AI feature into your current job — the best portfolio piece is a real one

Pick your primary cloud (do this in Week 1)

Option	Why pick it
Azure OpenAI	Most common in enterprises already on Microsoft; strong compliance story, private networking, GPT models
AWS Bedrock	Model choice (Anthropic Claude, others); fits AWS shops; good if your company is AWS-heavy

Either works with Spring AI and LangChain4j. Choose the one your target employers/current company use, and build everything on it so your portfolio matches real infra.

Weekly rhythm (10–15 hrs)

- **~4 hrs understanding** — Phase-0 fundamentals early; enterprise patterns later
- **~8 hrs building** — as real services, in whichever language fits the project (JVM or Python)
- **~1 hr notes** — Feynman test: explain the concept simply, or you don't have it yet

Core resources

Area	Resources
Understanding	Karpathy ("Intro to LLMs", "Let's build GPT"); 3Blue1Brown transformer series; Jay Alammar "Illustrated Transformer"; Chip Huyen *AI Engineering*
Building (JVM)	Spring AI reference docs; LangChain4j docs & examples
Building (Python)	LangChain / LlamaIndex / Pydantic AI; the Anthropic & OpenAI cookbooks (working code for every pattern)
Cloud (either language)	Azure OpenAI / AWS Bedrock docs; Anthropic "Building effective agents"
Enterprise angle	OWASP "Top 10 for LLM Applications" (security); Hamel Husain on evals; simonwillison.net for the running state of the field

Rules

1. **Understand the mechanism, then integrate it.** You should be able to explain *why*, not just wire it up.
2. **Every project is an integration**, built as a real service — not a notebook demo.
3. **Design for the enterprise from Phase 2 on:** security, cost, latency, failure. That's your moat.
4. **No frameworks-as-magic.** Spring AI, LangChain4j, LangChain — all fine, in any language, but know what they abstract (that's why Phases 1–2 stay close to the metal). Pick the stack per project, not by loyalty.

5. **Everything eventually gets an eval + a cost number.** "It seems to work" doesn't survive a design review.
